PicoCTF 2026 - MultiCode

MultiCode

Easy | General Skills | picoCTF 2026

AUTHOR: YAHAYA MEDDY

Description

We intercepted a suspiciously encoded message, but it's clearly hiding a flag. No encryption, just multiple layers of obfuscation. Can you peel back the layers and reveal the truth?

In this challenge, we're asked to decode multiple layers of encoding to read the flag

PicoCTF 2026 – MultiCode Base64 Encoding

```
└─$ cat message.txt
NjM3NjcwNjI1MDQ3NTMyNTM3
```

The first layer of encoding in this message is Base64 encoding

PicoCTF 2026 – MultiCode Base64 Encoding

```
Base64 Character Set

ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmn
opqrstuvwxyz0123456789+/
```

This encoding system takes data and encodes it into a text string with uppercase / lowercase letters, numbers and a couple of special characters, ' + ' and ' / '

PicoCTF 2026 – MultiCode Base64 Encoding

```
└─$ echo -n 'this is the unencoded text' | base64
dGhpcyBpcyB0aGUgdW5lbmNvZGVkIHRleHQ=
```

```
└─$ echo -n 'dGhpcyBpcyB0aGUgdW5lbmNvZGVkIHRleHQ=' | base64 -d
this is the unencoded text
```

We can use the Linux base64 command to decode the string

PicoCTF 2026 – MultiCode Hexadecimal ASCII Encoding

```
└─$ echo -n 'this is the unencoded text' | base64
dGhpcyBpcyB0aGUgdW5lbmNvZGVkIHRleHQ=
```

```
└─$ echo -n 'dGhpcyBpcyB0aGUgdW5lbmNvZGVkIHRleHQ=' | base64 -d
this is the unencoded text
```

The resulting output is a string of numbers and letters, which appears to be hexadecimal digits. This points to hexadecimal ASCII encoding

PicoCTF 2026 – MultiCode Hexadecimal ASCII Encoding

```
└─$ echo -n 'ABC' | xxd -p
414243

┌──([redacted])-[/tmp]
└─$ echo -n '414243' | xxd -p -r
ABC
```

ASCII encoding is an encoding system where strings of numbers are converted to ASCII characters and vice versa

PicoCTF 2026 – MultiCode Hexadecimal ASCII Encoding

```
└─$ echo -n 'ABC' | xxd -p
414243

┌──(thruhyhat㉿hacherfrogs)-[/tmp]
└─$ echo -n '414243' | xxd -p -r
ABC
```

Using Linux, we can convert hexadecimal digits to ASCII and vice versa with the xxd program

PicoCTF 2026 – MultiCode Hexadecimal ASCII Encoding

```
└─$ echo -n 'ABC' | xxd -p
414243

┌──(thruhyhat㉿hacherfrogs)-[/tmp]
└─$ echo -n '414243' | xxd -p -r
ABC
```

Using Linux, we can convert hexadecimal digits to ASCII and vice versa with the xxd program

PicoCTF 2026 – MultiCode URL Encoding

```
└─$ cat message.txt | base64 -d | xxd -r -p
cvpbPGS%7Barfgrq_rap0qvat_8qq03rsr%7D
```

Now the output string looks like it's been ROT13 encrypted, but there's also some URL encoded characters in the string

PicoCTF 2026 – MultiCode
URL Encoding

- original: `http://example.com/my file.txt`
- encoded: `http://example.com/my%20file.txt`

URL encoding is applied to web addresses, and transforms characters that are usually not allowed in URLs (special characters like $, [ ], ', etc..), into a % symbol followed by a two hexadecimal digits

PicoCTF 2026 – MultiCode URL Encoding

```
ASCII Hexadecimal to URL Encoding

0×41 ⟶ %41 ⟶ A

0×20 ⟶ %20 ⟶ ' ' (space)
```

The two hexadecimal digits used with the percentage sign in URL encoding map directly to the hexadecimal digits for ASCII encoding

PicoCTF 2026 – MultiCode URL Encoding

```
python3 -c
"import sys, urllib.parse;
print(urllib.parse.
unquote(sys.stdin.read().strip()))"
```

On Linux, we can use Python to URL decode special characters back into their original forms

PicoCTF 2026 – MultiCode Flag Format Inference

```
└─$ cat message.txt | base64 -d | xxd -r
rse.unquote(sys.stdin.read().strip()))"
cvpbPGS{arfgrq_rap0qvat_8qq03rsr}
```

The resulting output looks like a CTF that has been encrypted with a substitution cipher? How do we know? The format

PicoCTF 2026 – MultiCode Flag Format Inference

```
└─$ cat message.txt | base64 -d | xxd -r
rse.unquote(sys.stdin.read().strip()))"
cvpbPGS{arfgrq_rap0qvat_8qq03rsr}
```

All Pico CTF flags start with picoCTF, followed by some contents between a pair of curly braces, which is very similar to what we see here

PicoCTF 2026 – MultiCode ROT13 Encryption

```
ROT13 Encryption

A B C D E F G H I J K L M
| | | | | | | | | | | | |
N O P Q R S T U V W X Y Z
```

The most common substitution cipher used in cybersecurity challenges is ROT13, the simplest application of the Caesar cipher

PicoCTF 2026 – MultiCode ROT13 Encryption

```
ROT13 Encryption

A B C D E F G H I J K L M
| | | | | | | | | | | | |
N O P Q R S T U V W X Y Z
```

In ROT13 encryption, each letter of the ciphertext (secret message) is rotated 13 positions in the alphabet to form the plaintext (original message), e.g, the letters ABC would become the letters NOP, and vice versa

PicoCTF 2026 – MultiCode ROT13 Encryption

```
tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

In Linux, we can apply the tr command to apply the ROT13 encryption / decryption operation and get the fully decoded flag